Creating a Knowledge-based Recommendation System for Fragrance Selection

Submitted by: Archit Agarwal, Yue Yang

MSDS 459: Knowledge Engineering

Dec 1st, 2023

Abstract

The fragrance industry, known for its rich varieties and sophisticated selection processes, can greatly benefit from technological advancements. This report introduces building a fragrance recommendation system based on knowledge graph. This system is designed for the fragrance and retail industry, utilizing the power of the latest Large Language Models (LLMs) like ChatGPT, allowing customers to search quickly and intuitively to discover fragrances that most contextually relevant to what they are looking for. Through a clean web interface, users can access and search from a vast database of fragrance profiles, reviews, and ingredient information. This recommendation system is set to simplify customers' decision-making process and provide personalized scent recommendations. This innovation offers customer an AI-powered browsing and shopping experience, also is in line with the needs and preferences of modern consumers, which is a significant step towards technology transformation.

Introduction

Across the globe, the fragrance industry is a burgeoning market, with a projected value of approximately $52 billion by 2025 (Grand View Research, 2021, 5). This industry encompasses a wide range of products, from personal perfumes to home scents, with major players like Chanel and Dior dominating the market (Statista 2022, 30).

In the context of consumer behavior, selecting the right fragrance is often a complex and personal decision. Research indicates that nearly 60% of consumers find it challenging to choose a fragrance that aligns with their preferences and personality (Journal of Consumer Research 2023, 7). To delve deeper into the consumer decision-making process in fragrance selection, I explored various factors influencing their choices. The key considerations include:

1.Personal scent preferences and associations

2.The influence of branding and marketing on perception

While personal preference is paramount, the impact of marketing and brand image cannot be understated in fragrance selection. Another aspect to consider is the information overload consumers face. With thousands of fragrances available, making an informed choice can be overwhelming.

This complexity in fragrance selection underscores the need for a knowledge-based recommendation system. Such a tool would not only aid consumers in navigating the vast array of choices but also enhance their shopping experience by aligning recommendations with their personal preferences and scent profiles.

A fragrance selection recommendation engine would be a valuable asset not just for consumers but also for retailers and manufacturers. The fragrance industry, like many others, has a wealth of data on consumer preferences, scent compositions, and market trends. This data can be leveraged to create a sophisticated search engine powered by AI and machine learning algorithms, capable of providing personalized fragrance recommendations based on individual user inputs and preferences.

Literature Review

A crew chatbot is an emerging QSR industry demand actively discussed amongst the entire executive band. The chatbot needs to have the following characteristics to help the crew members:

1. Conversational capabilities
2. Context understanding
3. Feedback loop
4. Multilingual support

Conversational capabilities and context understanding:

Chronologically, the initial developments of the chatbot started around the 1970s in the early Internet age, when most chatbots were rule-based engines. The early chatbots are usually rule-based, social chatbots designed to return a predefined message when questioned with a keyword (Mnasri 2019,1-2). Although such chatbots are effective in a general context, they might not work with the crew-training use case.

The immediate next step to 1970s chatbots was Q-A chatbots trained on a question-answer database. While such chatbots are easy to build and deploy, managing content is nearly impossible. The QSR industry works at scale, and the questions vary from region to region. Also, compiling a list of Q/A is complex, time-consuming and costly. Further, the process of compilation needs to be automated. Hence, such chatbots will not fit the requirement as well.

Hence, with advancements in computing capacity, the data-based chatbots that use complex machine learning algorithms are a great success in Q/A use cases. However, they need help to handle complex user queries or support context understanding. Although quite old, most of the QSR industry employs chatbots as their initial layer for customer feedback. As part of the study, we focus on the top two QSR firms, Starbucks and McDonald's.

Following are examples:

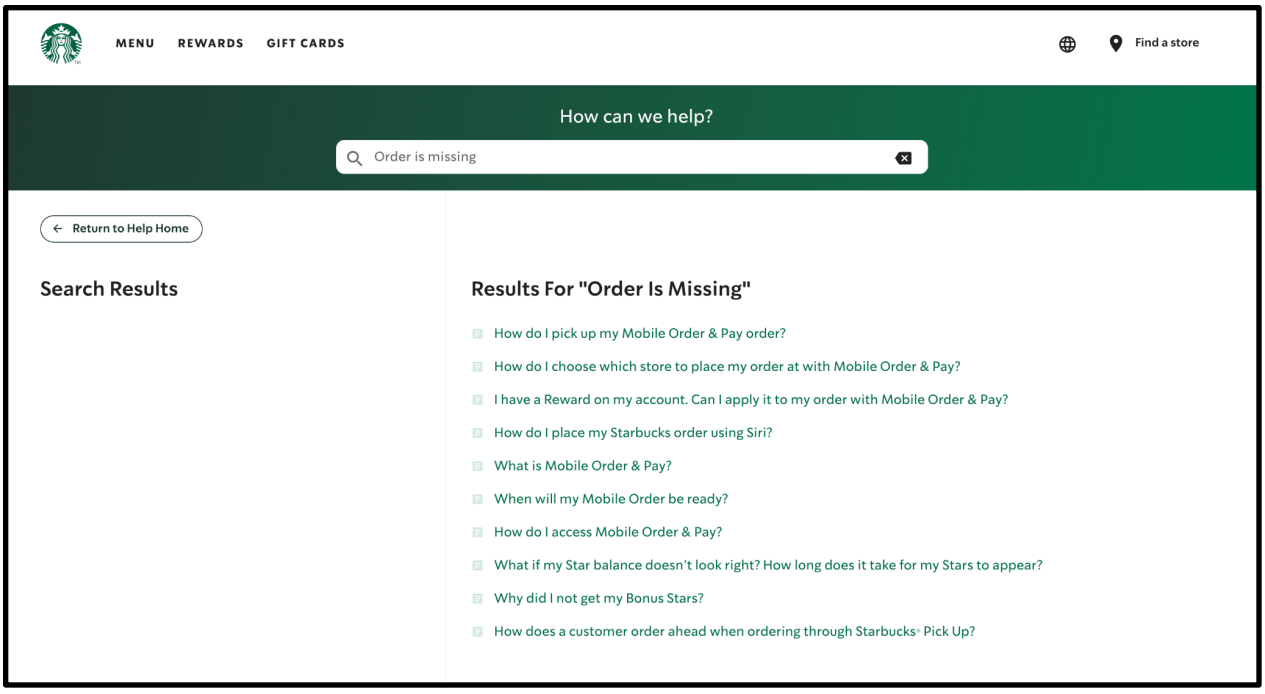


Fig 1: Starbucks *'Help Me'* Chatbot (www.starbucks.com)

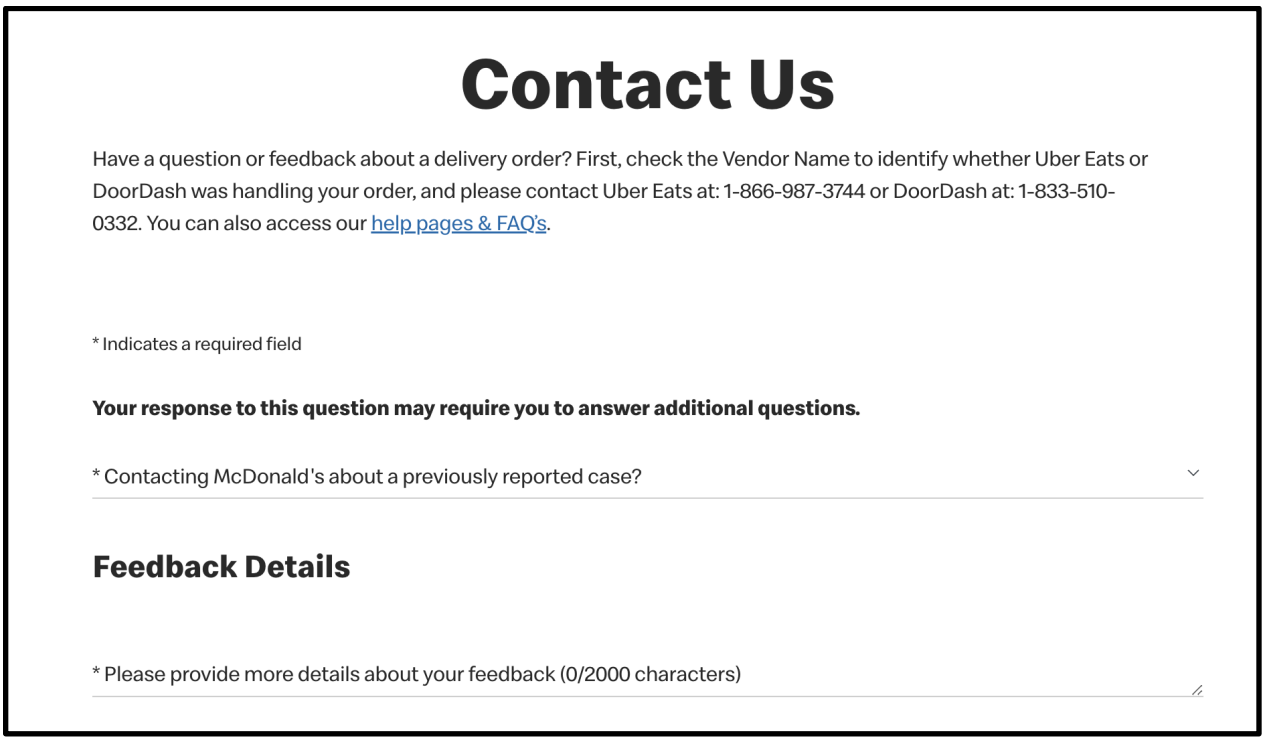


Fig 2: McDonald's *'Help Me'* Page (www.mcdonalds.com)

Based on the initial research about new chatbot technologies post-invention of Reinforcement Learning, it became evident that new advancements in GPT technology, such as GPT 3 and GPT 3.5, provide excellent support for conversational skills. Further, such technologies also support understanding content and personalizing the response based on a prompt (Chen et al. 2023).

GPTs belong to the class of Large Language Models, which are trained on nearly all available internet text. Moreover, major technology organizations have been racing towards developing the most potent GPT models, giving rise to the LLM market. Amongst all the available models, da-vinci-003 (GTP 3.5 series) is the most robust, exhibits advanced numerical sensitivity and performs better in terms of few-shot scenarios i.e. semantic matching tasks (Chen et al. 2023).

Further, with recent advancements in GPT accelerator models such as Langchain, one can efficiently use the GPT 3 and GPT 3.5 model as a Q/A assistant as it provides specific classes such as Agents and memory that enable context storage and to-and-fro response to the GPT 3.5 model.

GPT 3.5 functions on Reinforcement Learning, specifically transformers. The transformers comprise encoder and decoder layers that minimize the distance between the original and the post-encoded-decoded value (Caelen 2023). A significant advantage of the transformers is that they can accommodate multiple modalities such as image, video, text and speech.

Following is the architecture of how transformers work (Vaswani et al. 2017):

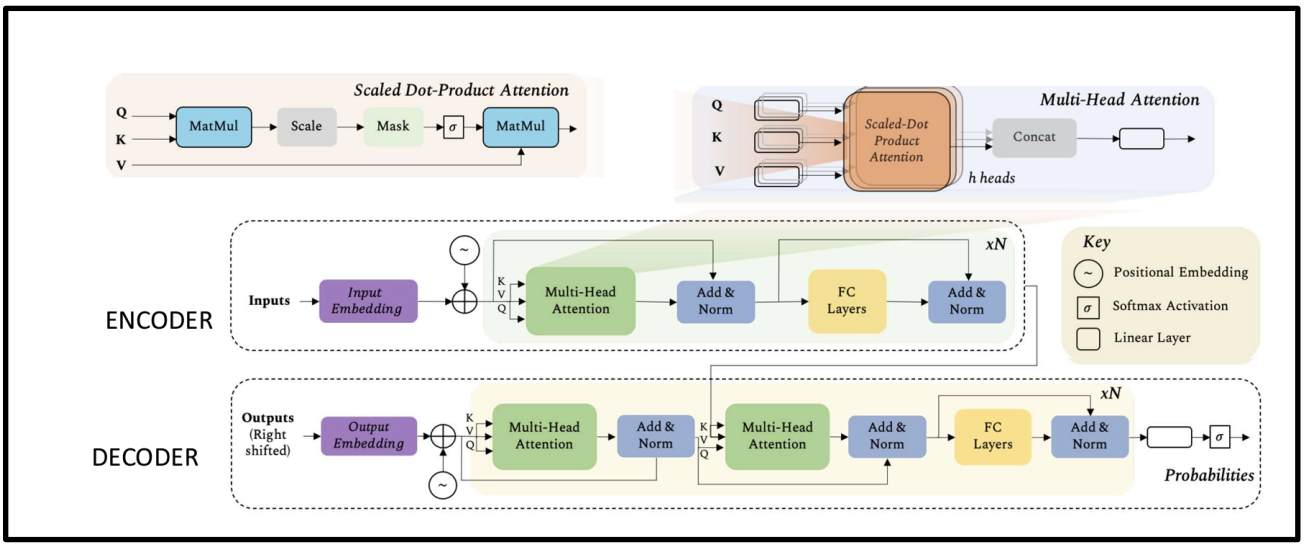


Fig 3: Encoder-Decoder Transformer (https://www.crcv.ucf.edu/wp-content/uploads/2018/11/Lecture-2-TransformerIntroductionFinal.pdf)

The GPT 3.5, in sync with the Langchain model, can be used to create a conversational and contextual chatbot. A simple version of Microsoft AI (text-davinci-002) is shown below responding to a user query on *"Three reasons to start a succulent garden":*

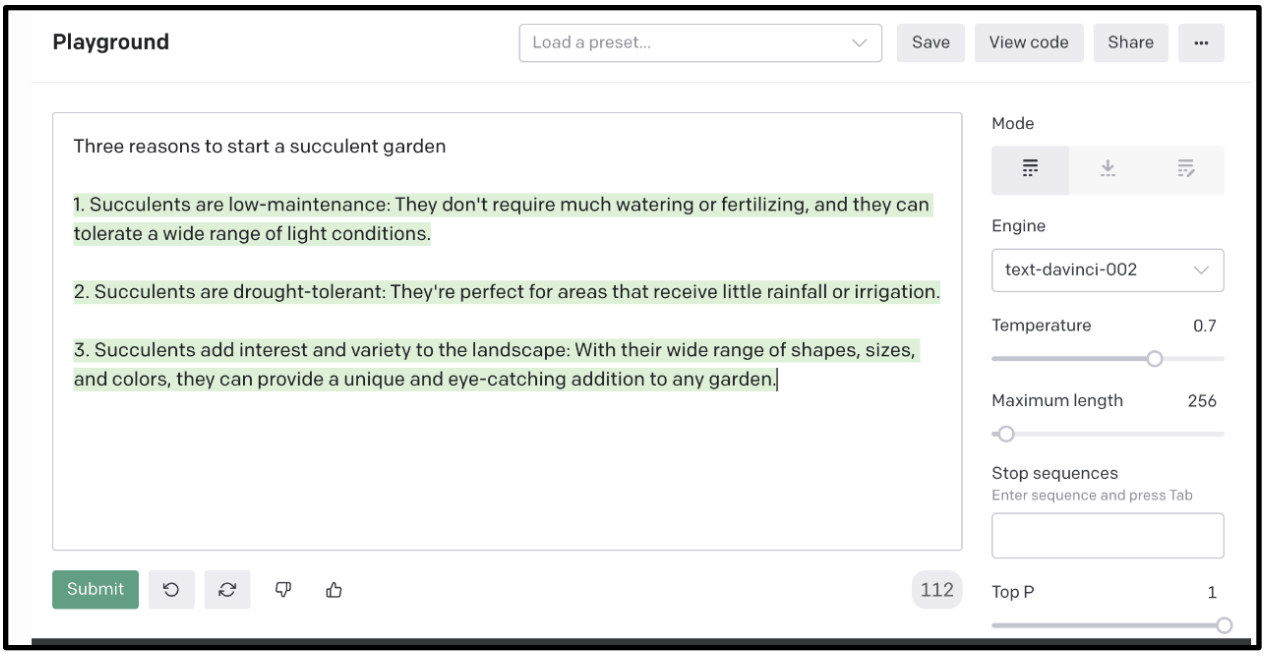


Fig 4: Microsoft AI Suite Chatbot window (https://simonwillison.net/2022/Jun/5/play-with-gpt3/)

At the core of the above dialogue, it is performing the following tasks:

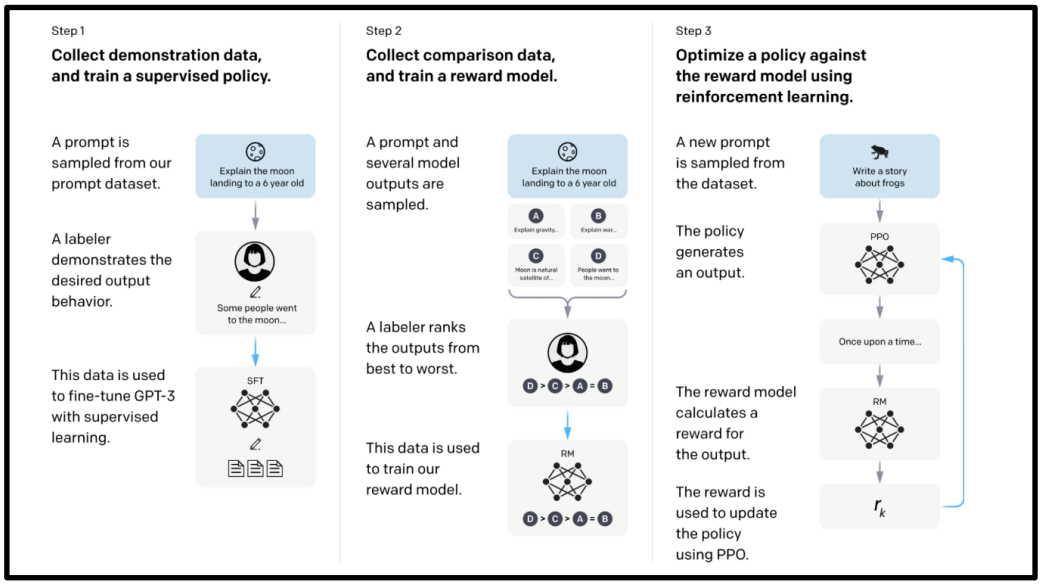


Fig 5: OpenAI GPT 3.5 process flow (https://openai.com/blog/chatgpt)

In the above diagram, the three steps are independent but simultaneously linked via the reward model. Firstly, the GPT-3 model is fine-tuned using supervised learning, where prompts and corresponding ideal responses are collected from human labelers to create a training set. This fine-tuned model is referred to as the SFT model.

Next, the text explains the RLHF (Reinforcement Learning from Human Feedback) stage, which consists of two steps. In the first step, a reward model is built to automatically score responses based on their alignment with the given prompts. Human labelers rank multiple responses generated by the SFT model to create a dataset for training the reward model.

In the second step of RLHF, reinforcement learning is employed. The initial SFT model is used as the generative model, and the process begins by selecting a random prompt and generating an output. The reward model then evaluates the output, and based on the received reward, the generative model is updated. This iterative process is repeated multiple times to improve the model's performance without human intervention. The final result is the GPT model, which is better at producing accurate completions based on the input provided in the prompt[7].

The Retrieval-Augmented Generation(RAG) is an exciting concept that enables chatbots to understand the context. RAG presents a paradigm that cleverly combines the generative capabilities of large-scale neural language models with extractive retrieval systems, aiming to produce more detailed and contextually enriched outputs. Large neural language models like GPT (Radford et al., 2019) have demonstrated impressive capabilities in generating coherent and contextually relevant text across various tasks. However, they rely solely on their internal representations to generate responses, which might limit their ability to pull specific or nuanced information from extensive corpora—retrieval systems, which can extract relevant snippets of information from large datasets.

The RAG system operates in two primary stages: a retrieval step and a generation step (Lewis et al., 2020).

1. Retrieval Step: First, given a query or prompt, the system searches through a vast corpus of documents to retrieve relevant passages or snippets of information. It is typically achieved using a dense retrieval mechanism, where both the query and the documents are embedded into a high-dimensional space, and the most relevant documents are retrieved based on their proximity to the query.
2. Generation Step: The retrieved documents (or their representations) are combined with the original query and fed into a sequence-to-sequence model, generating a coherent and detailed response.

This two-step mechanism allows the model to harness the specific, factual information from the retrieved documents, while the generative model ensures the response is coherent, contextually appropriate, and fluent. The strength of RAG lies in its ability to incorporate external knowledge into the generative process. This is particularly beneficial for tasks requiring factual accuracy and specificity; the retrieval step ensures that the response is grounded in reliable sources. For queries about niche topics or specifics that might not be sufficiently covered in the training data of a standalone language model, RAG can pull from the external corpus to provide detailed answers.

RAG has achieved state-of-the-art results on various question-answering benchmarks compared to standalone language models or purely retrieval-based systems. Its hybrid nature allows it to harness the best of both worlds: retrieval systems' specificity and accuracy and generative models' fluency and coherence.

However, The two-step process can introduce latency, especially if the retrieval step involves searching through massive corpora. The effectiveness of RAG is contingent upon the quality of the retrieved documents. Only accurate retrievals can lead to correct or irrelevant generation. Training RAG models end-to-end requires specialized infrastructure and techniques, given the need to juggle both the retrieval and generation components.

Feedback Loop and Multilingual Support:

Another essential feature of the GPT AI model is a Feedback loop that enables it to learn from its mistakes by understanding the failed conversation. A failed conversation will be one in which the user identifies answers given by the GPT3/3.5 model as irrelevant or out of context. Researchers define an out-of-context answer as a hallucination[8]. Since the GPT models are generative, i.e., they can generate a relevant interaction based on the training data, they can similarly put all out-of-context ideas as a response to a user question. Such feedback reports are crucial to retrain the GPT model to account for anomalies.

Finally, a GPT model can account for multilingual support. The GPT 3.5 model can answer any question in nearly 90+ languages. The model works on POS tagging the tokens; therefore, it can identify the context from the tagged token. Given the identified context, the model can identify similar contextual words in another language. Post identification, it can, therefore, transcribe in a new language. Following is the list of languages currently supported by the GPT 3.5 model that can cover most of the natural languages:

Language	Code	Pop. (M)	CC Size (%)	Cat.
English	en	1,452	45.8786	H
Russian	ru	258	5.9692	H
German	de	134	5.8811	H
Chinese	zh	1,118	4.8747	H
Japanese	jp	125	4.7884	H
French	fr	274	4.7254	H
Spanish	es	548	4.4690	H
Italian	it	68	2.5712	H
Dutch	nl	30	2.0585	H
Polish	pl	45	1.6636	H
Portuguese	pt	257	1.1505	H
Vietnamese	vi	85	1.0299	H
Turkish	tr	88	0.8439	M
Indonesian	id	199	0.7991	M
Swedish	sv	13	0.6969	M
Arabic	ar	274	0.6658	M
Persian	fa	130	0.6582	M
Korean	ko	81	0.6498	M
Greek	el	13	0.5870	M
Thai	th	60	0.4143	M
Ukrainian	uk	33	0.3304	M
Bulgarian	bg	8	0.2900	M
Hindi	hi	602	0.1588	M
Bengali	bn	272	0.0930	L
Tamil	ta	86	0.0446	L
Urdu	ur	231	0.0274	L
Malayalam	ml	36	0.0222	L
Marathi	mr	99	0.0213	L
Telugu	te	95	0.0183	L
Gujarati	gu	62	0.0126	L
Burmese	my	33	0.0126	L
Kannada	kn	64	0.0122	L
Swahili	sw	71	0.0077	X
Punjabi	pa	113	0.0061	X
Kyrgyz	ky	5	0.0049	X
Odia	or	39	0.0044	X
Assamesese	as	15	0.0025	X

Fig 6 Languages supported by GPT 3.5 (https://openai.com/blog/chatgpt)

Data

For the data, we collated a list of perfume websites available for free use. This data collection is pivotal in creating a comprehensive system that can accurately recommend fragrances based on individual preferences.

Our primary data source is Fragrantica, a renowned fragrance website. This platform hosts an extensive database, including nearly 85,000 perfumes, of which around 2,500 are actively marketed. The site also features over 100,000 user reviews for these perfumes, providing a rich source of consumer insights and preferences. The key sections of Fragrantica we utilize include:

1. Designer information (fragrantica.com/designers)

2. Fragrance notes (fragrantica.com/notes)

3. User profiles and reviews (fragrantica.com/users)

4. Fragrance groups (fragrantica.com/groups)

To supplement, we are also utilizing Parfumo, another valuable resource in the fragrance industry. This site will help fill in gaps in our data, particularly regarding specific brands and fragrance characteristics. The sections we are accessing include:

1. Fragrance classification by occasion (parfumo.com/Discover/Classification/Occasion)

2. Classification by season (parfumo.com/Discover/Classification/Season)

3. Target audience (parfumo.com/Discover/Classification/Audience)

4. Fragrance types (parfumo.com/Discover/Classification/Type)

5. Longevity of fragrances (parfumo.com/Discover/Longevity)

6. Sillage information (parfumo.com/Discover/Sillage)

Additionally, we utilize Wikiparfum (Wikiparfum – Ingredients) to gather specific data on the method of preparation and origin country of scent types, which helps us understanding the unique characteristics of each fragrance.

Our collected data encompasses a wide range of information for our recommendation system, including details on fragrance notes, longevity, sillage, user preferences, and reviews. The focus of our system is global, with a particular emphasis on markets with a high demand for fragrances.

While the primary language of our dataset is English, we are also considering multilingual capabilities to cater to a diverse user base. This approach will enhance the accessibility and usability of our system across different regions and language speakers.

The following cloud location contains all the scrapped data from the above links:

https://docs.google.com/spreadsheets/d/1-aCKSngW9EU3MnAmmpffXFLDLsZdBU0M2WvsYX6oZKc /edit#gid=0

Methods

Data Cleaning and Preparation:

The corpus contains many web links and irrelevant sections that might lead to hallucinations. Hence, we would remove the web links manually. Further, the data will be ingested as PDF files via the LangChain model HuggingFace API. The LangChain model LLM class will ingest the data and create relevant files for the immediate following workflow.

Model Processing Workflow:

Post ingestion, the Agents class of the LangChain model will ingest the PDFs and convert them into VectorDB. A vector database is the vectorized representation of all the tokens that are available in PDFs, such as "*griddle*," *"processing," and "food,"* amongst others. A vectorized database is an input to the LLM model, such as GPT 3/3.5, which will ingest such prompts and curate the responses based on the learned data. Hence, the GPT model now works as a *"document-brain"* for the crew members as it understands the PDFs in-out.

To convert this model as a chatbot, the LangChain model and HuggingFace APIs provide a rich interaction tool that can be mounted on top of the GPT model, connecting the crew- members directly to the *"document brain."*

The crew members can directly interact with the interaction tool by authenticating using their email IDs and password. Post authentication, they can interact with the chat-based window that can help answer crew questions. Also, by posting an answer from the GPT model, the crew member can provide feedback if they are satisfied with the answer or dissatisfied with the response. As we need to improve the contextual understanding of the chatbot, we will be using RAG architecture for the chatbot.

The model will also store the conversation as part of the cache for predefined messages; after that, the model memory will reset to default. A schematic of the entire process flow is presented below:

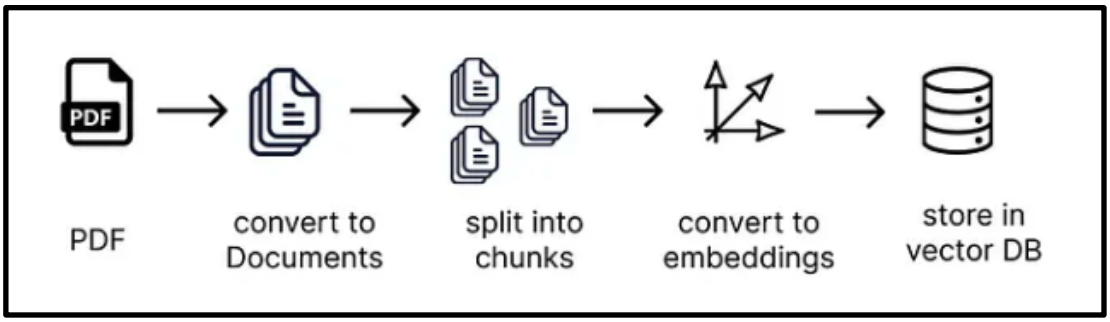


Fig 7: Step 1: Data Engineering Pipeline

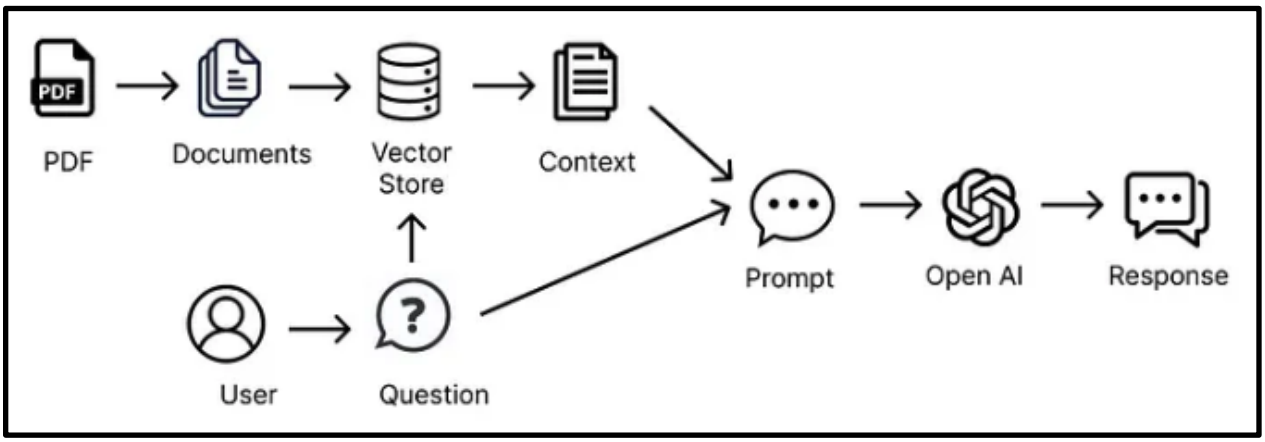


Fig 8: Using RAG to generate chatbot response

Model Building:

We need to code/create the above workflow for Model building. The PDF in 'Fig 8' refers to the data collected from online sources. For the first attempt, we started with the Burger King data that contains four PDF files combining technical and cleaning information about the Shake and Soft Serve Machines, specifically the Taylor Models 632, C707, C709, and C606.

Based on the research, we identified the best possible infrastructure based on the pricing of each component and availability. We used *ChromaDB* for the vector database that stored all the embeddings from the PDFs. Next, we used the OpenAI API to use GPT 3.5 Turbo 32K model as our base LLM. Next, we designed a quick Python interface to interact with the chatbot.

We started model building in Jupyter Notebook, hosted on a local PC that is Mac OS 13 Ventura with GPU that enables quick model development. As a preliminary requirement, we installed the following packages:

1. Langchain
2. OS

3. Sys
4. ChromaDB
5. OpenAI
6. TikToken

All the packages are available from the latest Python repository. We coded our entire project in Python 3.10.11, where we were required to install all these packages. The data resided in the same directory as the Jupyter Notebook we used to create the chatbot. We labeled the data repository as -' Data' and then added all the documents from the Burger King and McDonald's repositories to the data folder.

Next, we added an API key that connects the Jupyter environment to the OpenAI model GPT 3.5. Once everything was added to the Jupyter notebook, the following agenda is just to ingest the PDFs and convert them into chunks; we used the CharacterTextSplitter function from Langchain that split the PDFs and created chunks of 800 characters that are called documents in the model building context.

Next, we ingest these documents into a vector data-ChromaDB that creates a 3D representation of all the embeddings ingested from documents. Following is a schematic representation of some of the words hungry, food, and bicycle:

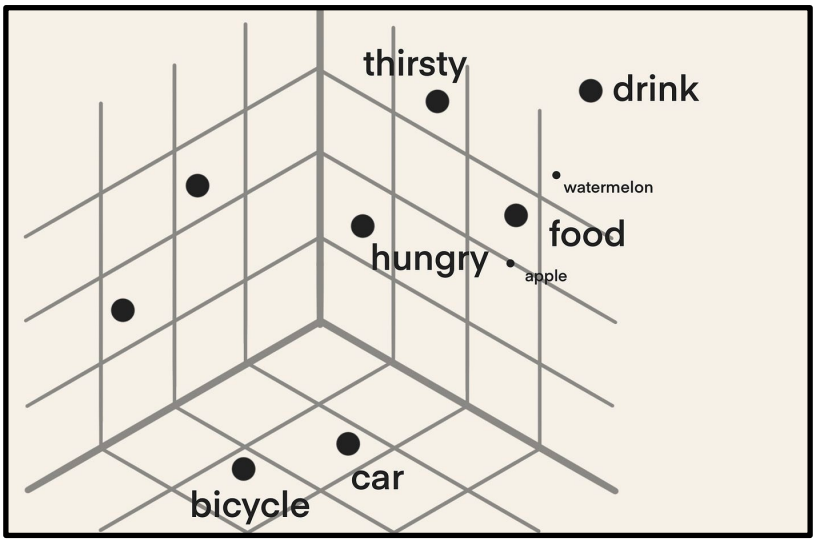


Fig 9: Schematic representation of words in 3D space as vectors
(https://towardsdatascience.com/explaining-vector-databases-in-3-levels-of-difficulty-fc392e48ab78)

The vector database would be input along with the question from the user out of context- a crew member in QSR to the GPT 3.5. The model will identify embeddings from the question and search them in the 3D space created during training. It is very similar to how Google Search works in general. The returned embeddings from the model's query to the vector database are then used to create the customer's answer.

Several hyperparameters are used within the model to tune it, such as:

1. Temperature (A higher temperature value typically makes the output more diverse and creative but might also increase its likelihood of straying from the context.)
2. Top P: The number of top searches used by model
3. Top K: The number of keywords used by the model to search

Evaluation:

Although a Generative AI model such as GPT 3/3.5 provides a generative response that has yet to be seen anywhere, it becomes complex to evaluate the response. Therefore, a user-feedback loop is an essential part of validation; engineering validation is also required before launch to crew members. Cosine Similarity is the most widely used metric if a human response is available against the question. Since, most of the time, such answers may not be available, we will minimize the temperature setting in GPT models that controls the degree of *"generative"* content by the model. We use metrics such as the BLEU and ROGUE scores to evaluate the model from the human answer. However, the current scope is to identify if the model can understand the context and return an answer about the context. Hence, we evaluated the model from our qualitative perspective of answer and not a quantitative metric.

Results

The codework compiled and created a chat window through which we could interact with the chatbot and test it out for many questions. As part of the model building and training, we created five chatbots, four specialized in each mega-QSR, one each for Starbucks, Wendy's, Burger King, and McDonald's. Here are some of the examples of chats that we had with the Burger King chatbot:

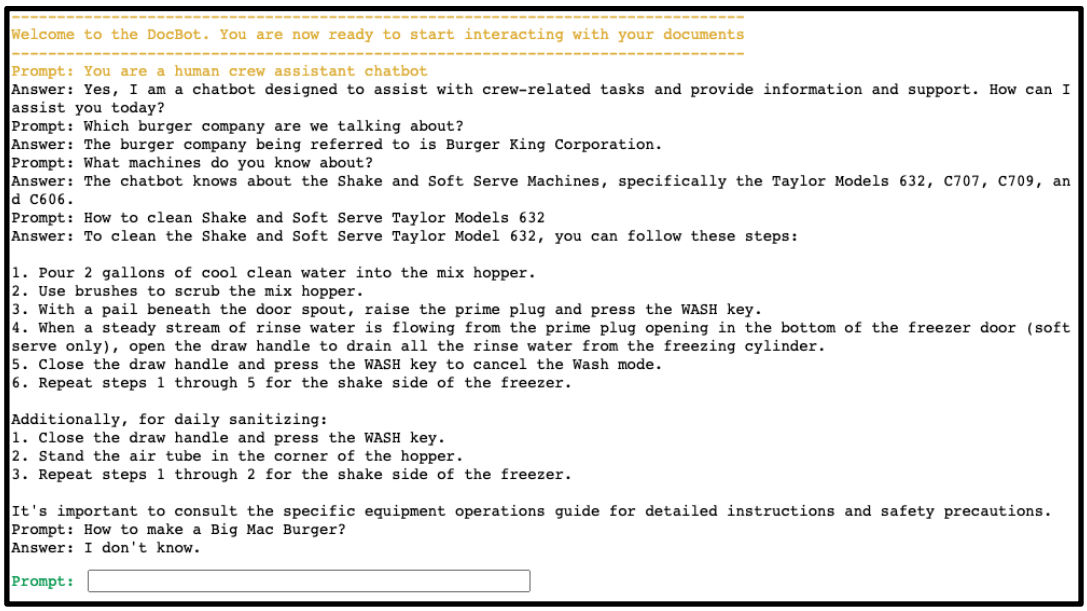


Fig 10: Chat results with Burger King crew chatbot

Furthermore, the following are the chat windows from each of the other chatbots:

1. McDonald's Chatbot:

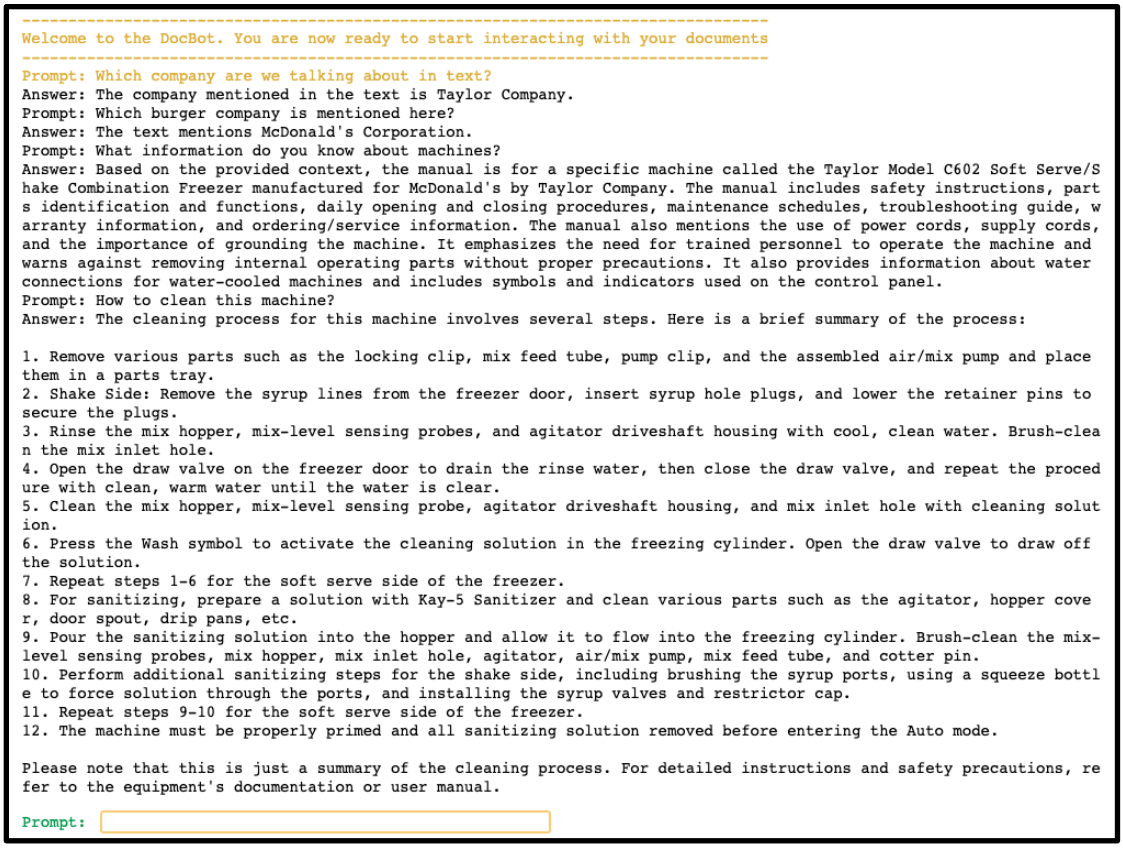


Fig 11: Chat results with McDonald's crew chatbot

2. Wendy's Chatbot:

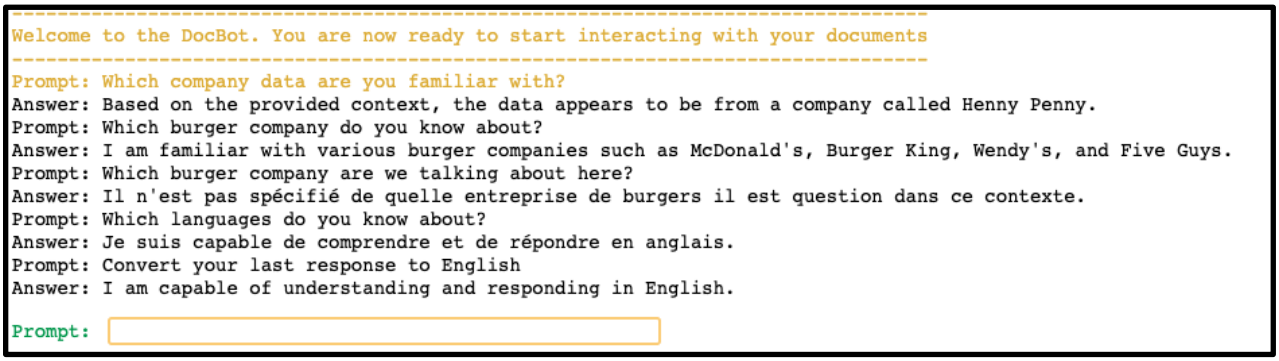


Fig 12: Chat results with Wendy's chatbot

3. Starbucks Chatbot:

```
--------------------------------------------------------------------------------
Welcome to the DocBot. You are now ready to start interacting with your documents
--------------------------------------------------------------------------------
Prompt: Which company are we talking about?
Answer: The company mentioned in the context is Thermoplan AG.
Prompt: How to clean the machine?
Answer: To clean the machine, follow these steps:

1. Carefully remove and empty the milk container(s).
2. Press the cleaning button and hold it for 3 seconds to start the cleaning process. To stop cleaning, press any oth
er button.
3. Wipe off both milk suction tubes using a fresh single-use disposable tissue or paper towel for each tube.
4. Clean the milk container(s) in a dishwasher and set them aside.
5. Empty the capsule container daily or when prompted.
6. After the cleaning process, remove the cleaning key and insert two cleaning tablets.
7. Place the empty 5L milk container in the refrigerator and insert both suction tubes into the container. Press any
button.
8. After cleaning, carefully wipe off both milk suction tubes using a fresh single-use disposable tissue or paper tow
el for each tube.
9. Thoroughly clean the inside walls of the refrigerator with potable water.
10. When the cleaning process is complete, remove the cleaning key and reinsert it to restart the machine from maximu
m energy saving mode.
11. Pour out the remaining water and clean the 5L milk container in a dishwasher.
12. Put the empty milk container(s) back in the refrigerator.

Note: Make sure to follow the specific instructions provided by the manufacturer for your machine.
Prompt: Is this guideline true for McDonald's as well?
Answer: No, this guideline does not specifically mention McDonald's machines. It is important to refer to the specifi
c guidelines and instructions provided by the manufacturer of the McDonald's machines for proper usage and maintenanc
e.
Prompt: Can you provide the relevant page for previous answer?
Answer: I'm sorry, but I don't have access to the specific pages of the document.

Prompt:
```

Fig 13: Chat results with Starbucks chatbot

As part of the exploration, we changed the questions and prompts across all four chatbots to evaluate whether the chatbot understands context. Also, we played around with all the hyperparameters mentioned above to see the impact on the chatbot's responses. The idea is to assimilate all the factors and identify the feasibility of creating a universal chatbot. Following are the results of qualitative research on these chatbots:

Chatbot Name	Burger King	McDonald's	Wendy	Starbucks
+Δ change in temprature	Accuracy ↑	Accuracy ↑	Accuracy ↓	Accuracy ↑
+Δ change in chunk size	Accuracy unchanged	Accuracy unchanged	Accuracy ↑	Accuracy ↑
GPT 3.5 vs GPT 4	Better answers	Better answers	Better answers	Better answers

Accuracy is defined as how accurately the chatbot is compared to the human answer. In the next section, we discuss these results in detail.

Analysis and Interpretation

General observations:

The chatbots are performing well based on the accuracy expectation from a quantitative perspective. All four chatbots have understood the crew questions and identified the relevant information in a low latency environment while maintaining accuracy. The chatbots were each able to maintain the conversational context as well. This is evident from using unclear antecedents such as 'this' while asking the questions.

The chatbots identified the meaning of unclear antecedents and exposed it to suitable embeddings, creating appropriate answers. Furthermore, the chatbots have been exposed to different data sets, whereby some were rich in contextual information while others were not. Hence, we were able to see the impact of contextual data. The contextual data for Wendy's was limited to just 1 PDF, while that of McDonald's was 6 PDFs. To illustrate the point, we picked responses from the Wendy's chatbot and McDonald's chatbot:

A response from Wendy's chatbot:

```
Prompt: Which burger company are we talking about here?
Answer: Il n'est pas spécifié de quelle entreprise de burgers il est question dans ce contexte.
```

vs. A response from McDonald's chatbot:

```
Prompt: Which burger company is mentioned here?
Answer: The text mentions McDonald's Corporation.
```

The chatbots changed the language in Wendy's case as the contextual data was unavailable, and the model started providing fabricated answers. In McDonald's case, the data was available, so the chatbot answered the question correctly. This phenomenon is called Hallucination, where the model fabricates its answer without any context and is controlled by a feature called temperature.

Impact of temperature:

The temperature setting goes from 0 to 1 and is a significant influencer of the creativity of the model. A temperature setting of 0 makes the model unpredictable in it is following outcome. We tried using 0 as a setting, and the answer did not make sense. Based on our analysis, the best setting would be nearly 0.96 as the model has some elements of creativity while it catered to the contextual information while fabricating the answers.

Impact of prompts:

We observed a significant change in the chatbot's behavior as we changed the input prompts. Note that the input prompts differ from those provided during the Q&A session with the chatbot. The input prompt defines the persona of the chatbot; while not crucial from a feasibility standpoint, the input prompt adds all the panache in the chatbot responses. The current input prompt was the following:

1. No input prompt: The chatbot responses were bland and sometimes pinching. It might come as offensive and blunt for a crew member working on the station.
2. “Act as a human customer service agent”: The responses were more humane and warming. Hence, this setting works better for the crew.

Similarly, we can try more such prompts until the crew feedback is highly optimistic on the way of communication of the chatbot. While input prompts influence the characteristics of the communication, it is a powerful tool in ensuring boundary cases, such as when the chatbot is unsure of the answer or intends to hallucinate.

Impact of the algorithm:

Another vital aspect impacting the chatbot outcome is the algorithms we passed as input features. We evaluated the performance on the same questions for GPT 3, GPT 3.5 Turbo, and GPT 4. The best responses come from GPT 4 when there is a sufficient amount of context, while it started to hallucinate more than GPT 3.5 with the same temperature setting and less amount of data. Once we added the same data and the same temperature, the performance of GPT 4 was far better than GPT 3/GPT 3.5.

This analysis provides us with some initial opinions about the impact of low data on the contextual training of chatbots.

Conclusions

The experiment provided great insights into the feasibility of the chatbot for the QSR industry. Per our understanding, there is great value in developing, tuning, and deploying such a solution at scale as it can understand the context of the question, engage the user in a chat, and remember the historical context.

Technology has challenges, but these challenges can be balanced with more exploration. Based on the initial work, we can identify the role of prompts, embedding creation, and algorithm as preliminary in continuing the exploration towards a better chatbot. Additionally, the impact of human understanding provided an excellent evaluation framework for the asked questions, but more is needed to provide a sustainable framework for future evaluations.

Directions for Future Work

The most crucial aspect is quantifying the evaluation metric using quantitative scores such as the BLEU score, ROGUE score, and average F1 score for the chatbot answers. Recent developments in RLHF (Reinforcement learning by human factor) have proven to be a remarkable strategy in evaluating generative AI chatbots.

Further, collecting more contextual data will improve the performance of the chatbots, given the proper experiment hyperparameters such as Temperature, Top K values, and Top P values.

A future experiment would start with collecting more data, identifying the quantitative values, determining the relevant algorithms, and then setting up the project work.

References

1. “Franchise Industry: QSR Employment US 2022.” n.d. Statista. Accessed July 17, 2023. https://www.statista.com/statistics/252290/estimated-employment-of-the-us-quick-service-restaurant-industry/#:~:text=In%202022%2C%20over%203.8%20million.

2. Statista. 2022. “Topic: McDonald’s.” Statista. November 23, 2022. https://www.statista.com/topics/1444/mcdonalds/#topicOverview.

3. Zippia. 2021. “Mcdonalds Crew Member Demographics.” Www.zippia.com. January 29, 2021. https://www.zippia.com/mcdonalds-crew-member-jobs/demographics/.

4. Mnasri, Maali. 2019. Review of Recent Advances in Conversational NLP: Towards the Standardization of Chatbot Building. ArXiv, March 1–5. https://arxiv.org/pdf/1903.09025.pdf.

5. Chen, Xuanting, Junjie Ye, Can Zu, Nuo Xu, Rui Zheng, Minlong Peng, Jie Zhou, Tao Gui, Qi Zhang, and Xuanjing Huan. 2023. Review of How Robust Is GPT-3.5 to Predecessors? A Comprehensive Study on Language Understanding Tasks. ArXiv, March 2–5. https://arxiv.org/pdf/2303.00293.pdf.

6. Caelen, Olivier, and Marie Alice Blete. 2023. Developing Apps with GPT-4 and ChatGPT. 1st ed. O’Riley Media INC. https://learning.oreilly.com/library/view/developing-apps-with/9781098152475/.

7. Vaswani, Ashish, Noam Shazeer, Niki Parmar, Llion Jones, Aidan Gomez, and Lukasz Kaiser. 2017. Review of Attention Is All You Need. Conference on Neural Information Processing Systems 31 (June): 2–4. https://arxiv.org/pdf/1706.03762.pdf.

8. Radford, A., J. Wu, Child, R., D. Luan, Amodei, D., and I. Sutskever. 2019. "Language Models are Unsupervised Multitask Learners." OpenAI. https://d4mucfpksywv.cloudfront.net/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

9. Lewis, P., Y. Liu, Goyal, N., M. Ghazvininejad, Mohamed, A., O. Levy, Stoyanov, V., and L. Zettlemoyer. 2020. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." arXiv preprint arXiv:2005.11401. https://arxiv.org/abs/2005.11401

Appendix:

1. Codework:

```
import os
From langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import CharacterTextSplitter
from langchain.vectorstores import Chroma
from langchain.embeddings import OpenAIEmbeddings
import sys
from langchain.document_loaders import Docx2txtLoader
from langchain.document_loaders import TextLoader
from langchain.vectorstores import Chroma
from langchain.llms import OpenAI
from langchain.chat_models import ChatOpenAI
from langchain.chains import ConversationalRetrievalChain
from langchain.memory import ConversationBufferMemory
from langchain.text_splitter import CharacterTextSplitter
from langchain.prompts import PromptTemplate
os.environ["OPENAI_API_KEY"] = "sk-XXX"# load the document as before

documents = []

path_init = "~/Desktop/NW-AI/Data/"

for file in os.listdir("Data/"):

    if file.endswith(".pdf"):

        pdf_path = path_init + file

        loader = PyPDFLoader(pdf_path)

        documents.extend(loader.load())

    elif file.endswith('.docx') or file.endswith('.doc'):

        doc_path = path_init + file

        loader = Docx2txtLoader(doc_path)

        documents.extend(loader.load())

    elif file.endswith('.txt'):

        text_path = path_init + file

        loader = TextLoader(text_path)ß

        documents.extend(loader.load())


# we split the data into chunks of 1,000 characters, with an overlap

# of 200 characters between the chunks, which helps to give better results

# and contain the context of the information between chunks

text_splitter = CharacterTextSplitter(chunk_size=800, chunk_overlap=200)
```

```
documents = text_splitter.split_documents(documents)
# we create our vectorDB, using the OpenAIEmbeddings tranformer to create
# embeddings from our text chunks. We set all the db information to be stored
# inside the ./data directory, so it doesn't clutter up our source files
vectordb = Chroma.from_documents(
  documents,
  embedding=OpenAIEmbeddings(),
  persist_directory="
)
vectordb.persist()
from langchain.chains import RetrievalQA
from langchain.llms import OpenAI

qa_chain = RetrievalQA.from_chain_type(
    llm=OpenAI(),
    retriever=vectordb.as_retriever(search_kwargs={'k': 7}),
    return_source_documents=True
)
pdf_qa = ConversationalRetrievalChain.from_llm(
    ChatOpenAI(temperature=0.7, model_name="gpt-3.5-turbo-16k"),
    vectordb.as_retriever(search_kwargs={'k': 6}),
    return_source_documents=True,
    verbose=False
)

yellow = "\033[0;33m"
green = "\033[0;32m"
white = "\033[0;39m"

chat_history = []
print(f"{yellow}----------------------------------------------------------------------------------")
print('Welcome to the DocBot. You are now ready to start interacting with your documents')
```

```
print('--------------------------------------------------------------------------------')
while True:
    query = input(f"{green}Prompt: ")
    if query == "exit" or query == "quit" or query == "q" or query == "f":
        print('Exiting')
        sys.exit()
    if query == '':
        continue
    result = pdf_qa(
        {"question": query, "chat_history": chat_history})
    print(f"{white}Answer: " + result["answer"])
    chat_history.append((query, result["answer"]))
```

2. UI Responses:

Welcome to the DocBot. You are now ready to start interacting with your documents

---

Prompt: Which company are we talking about?

Answer: The company mentioned in the context is Thermoplan AG.

Prompt: How to clean the machine?

Answer: To clean the machine, follow these steps:

1. Carefully remove and empty the milk container(s).

2. Press the cleaning button and hold it for 3 seconds to start the cleaning process. To stop cleaning, press any other button.

3. Wipe off both milk suction tubes using a fresh single-use disposable tissue or paper towel for each tube.

4. Clean the milk container(s) in a dishwasher and set them aside.

5. Empty the capsule container daily or when prompted.

6. After the cleaning process, remove the cleaning key and insert two cleaning tablets.

7. Place the empty 5L milk container in the refrigerator and insert both suction tubes into the container. Press any button.

8. After cleaning, carefully wipe off both milk suction tubes using a fresh single-use disposable tissue or paper towel for each tube.

9. Thoroughly clean the inside walls of the refrigerator with potable water.

10. When the cleaning process is complete, remove the cleaning key and reinsert it to restart the machine from maximum energy saving mode.

11. Pour out the remaining water and clean the 5L milk container in a dishwasher.

12. Put the empty milk container(s) back in the refrigerator.

Note: Make sure to follow the specific instructions provided by the manufacturer for your machine.

Prompt: Is this guideline true for McDonald's as well?

Answer: No, this guideline does not specifically mention McDonald's machines. It is important to refer to the specific guidelines and instructions provided by the manufacturer of the McDonald's machines for proper usage and maintenance.

Prompt: Can you provide the relevant page for previous answer?

Answer: I'm sorry, but I don't have access to the specific pages of the document.